Computer Science

Exercises

EXERCISE: AN ALGORITHM THAT SATISFIES: NUMS[I] - 10 = K

Suppose we have a list `nums` and a constant k.

You must find the first index `i` such that `nums[i] - 10 == k`. If there is no such index that satisfies the property, return `None`.

Example cases:

Case 1:
- **Inputs**: nums = [5, 20, 35, 23, 2, 5, 30, 6, 2, 3], k = 20
- **Output**: 6
- **Explanation**: nums[6] is 30, 30 - 10 = 20, and k = 20.

Case 2:
- **Inputs**: nums = [5, 2, 60, 50, 20], k = 15
- **Output**: None
- **Explanation**: There is no index i such that `nums[i] - 10 = k`.

1. Write a multi-step strategy towards solving the problem.
2. Write an algorithm to solve this problem.
3. Describe the upper-bound time and space complexity of your solution.

SOLUTION:

1.
   - We must make a function that accepts two parameters: `nums` and `k`.
   - Then we need to loop the `range(n)` where n is the length of `nums`, while accessing the index `i`, and for each `nums[i] - 10`, see if it is `==` to `k`.
   - If an index is found, and the property is satisfied, we immediately return the index and exit the function.
   - Otherwise, we keep looping, and if the loop ends and no index is found, we return `None`.

2.

```
def solution(nums, k):
  for i in range(len(nums)):
    if nums[i] - 10 == k:
      return i

  return None
```

3. The solution's time complexity is $O(n)$: Because the amount of instructions scale linearly ($y = ax + b$) with the input's size.
   The solution's space complexity is $O(1)$: Because we are only storing constant data regardless of input size.

EXERCISE: THE TWO SUM PROBLEM

Suppose we have a list `nums` and a constant `k`.

You must find two indices `i` and `j` such that `nums[i] + nums[j] = k` and $i \neq j$, if there are no such indices, return `None`.

Multiple solutions may be valid.

Example cases:

Case 1:
- **Inputs**: [3,2,4], k = 6
- **Outputs**: [1,2]
- **Explanation**: nums[1] + nums[2] == 6, we return [1,2] (or [2,1])

Case 2:
- **Inputs**: nums = [2,7,11,15], k = 9
- **Outputs**: [0, 1]
- **Explanation**: Because nums[0] + nums[1] == 9, we return [0, 1]

1. Write a multi-step strategy towards solving the problem.
2. Write an algorithm to solve this problem.
3. Describe the upper-bound time and space complexity of your solution.

SOLUTION:

1.
   - We must make a function that accepts two parameters: `nums` and `k`.
   - We will make a dict of numbers we have seen already, the keys will be numbers we've seen and the values will be indices.
   - We will loop over `nums`, accessing `i` and `nums[i]`.
   - Assuming we have `nums[i]`, `c = k - nums[i]` will be the number we need such that `nums[i] + c = k`, we will calculate that complement `c`.
   - If the complement is inside the dict, then we can return the index of the complement and `i`. (In other words, if the number c we need for `c + nums[i] = k` is inside the dict, we will return the indices of it and i).

2.
```python
def solution(nums, k):
    seen = {}

    for i in range(len(nums)):
        num = nums[i]
        complement = k - nums[i]

        if complement in seen:
            j = seen[complement]
            return [i, j]

        seen[num] = i

    return None
```

3. The time complexity of the algorithm is: $O(n)$
   The space complexity of the algorithm is: $O(n)$

SOLUTION:

1.
   - We must make a function that accepts two parameters: `nums` and `k`.
   - We will loop over `range(len(nums))` and access `i` and `nums[i]`.
   - For each `num` at `i`, we will loop over `range(i + 1, len(nums))`, to check the numbers in-front of `nums[i]` excluding `nums[i]` itself, from that loop we will access `j`.
   - In the previous loop, if `nums[i] + nums[j] == k`, then we can return `[i, j]`

2.
```python
def solution(nums, k):
  for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
      if nums[i] + nums[j] == k:
        return [i, j]

  return None
```

3. The time complexity of the algorithm is: $O(n^2)$: The function scales quadratically with its input.
   The space complexity of the algorithm is: $O(1)$: We are only storing constant data.

EXERCISE: APPROXIMATE PI USING MONTE CARLO METHODS

Monte Carlo methods are computational algorithms based on **repeated random sampling**, used to solve **deterministic** problems through **randomness**.

- **Area of a circle**: $a_c = \pi r^2$, where r is the radius.
- **Area of a square**: $a_t = a^2$, where a is the side-length of the square.
- **Equation of a circle**: $x^2 + y^2 = r^2$
- **Point $P(x_p, y_p)$ is inside the circle**: $x_p^2 + y_p^2 \leq r^2$
- **Point $P(x_p, y_p)$ is outside the circle**: $x_p^2 + y_p^2 > r^2$

Now, imagine a circle of radius r centered at $(0, 0)$, inscribed in a square, the square would have side-length $a = 2r$. The area of the square would then be $a_t = (2r)^2 = 4r^2$.

Now, imagine within the constraints of this square, we are randomly choosing n amount of points, and we will consider two cases:

- Points that landed in the circle
- Points that landed in the square

If you think about it, the proportion of points inside the circle approximates the proportion of the circle's area to the square's area. We will compute $\frac{a_c}{a_t}$ to get a formula for π.

$$\frac{a_c}{a_t} = \frac{\pi r^2}{4r^2} = \frac{\pi}{4}$$

Now, we will consider the amount of points that landed in the circle as the area of the circle (a_c), and the points that landed in the square as the area of the square (a_t), so we get:

$$\frac{\text{amount of points in the circle}}{\text{amount of points in the square}} \approx \frac{\pi}{4}$$

$$\pi \approx 4\left(\frac{\text{amount of points in the circle}}{\text{amount of points in the square}}\right)$$

The way we will resolve this equation using Monte Carlo is as following: for a number n, we will generate n points with random x and y coordinates, then, for each of these points, we will use the formulas to check if they landed in the circle or not, then we could resolve this equation.

Now, because `random.random()` only provides values between `0` and `1`, we will work with a quarter-circle (first-quadrant only) where: $0 \leq x \leq 1$ and $0 \leq y \leq 1$, which simplifies the process (no need to generate negative coordinates).

Here are the visuals: https://www.desmos.com/calculator/zmcv3rdppk

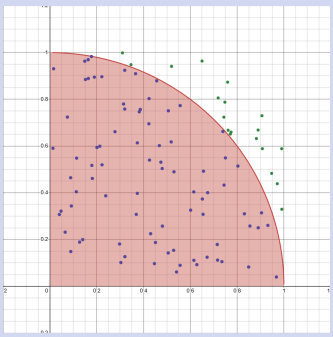

Figure 1: $n = 100$

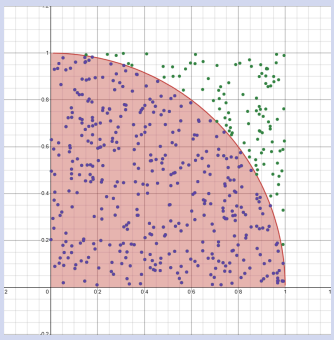

Figure 2: $n = 500$

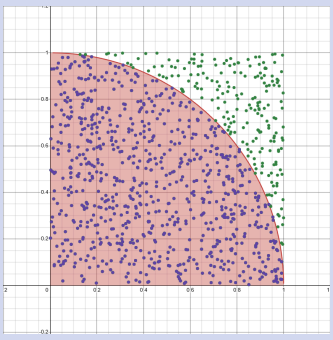

Figure 3: $n = 1000$

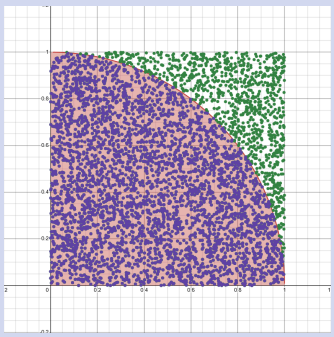

Figure 4: $n = 5000$

The purple points are the points *inside the circle*, and the green points are *outside the circle*.

1. Write a multi-step strategy towards solving the problem.
2. Write the algorithm to solve this problem.
3. Describe the upper-bound time and space complexity of your solution.

SOLUTION:

1.
   - We must make a function that takes a parameter: `amount_of_points`, the number of points to be generated.
   - We will consider arbitrarly that $r = 1 \text{ unit}$.
   - We will make a variable called `points_in_circle` to accumulate how many points landed in the circle.
   - We must loop `n` times.
   - For each loop, each time, we will generate a random `x_p` and `y_p` point that lies between 0 and 1 with `random.random()` from the `random` package.
   - We will then calculate `x_p**2 + y_p**2` for that point, and check if it is less than r^2, if it is, then it is inside the circle, and we will add `+1` point to `points_in_circle`.
2.

```python
import random

def solution(amount_of_points):
    points_in_circle = 0
    r = 1

    for i in range(n):
        x_p = random.random()
        y_p = random.random()

        distance = x_p**2 + y_p**2

        if distance <= r**2:
            points_in_circle += 1

    pi = 4 * (points_in_circle / amount_of_points)
    return pi
```

3. This algorithm has a time complexity of: $O(n)$: Because instructions increase linearly with input size. And has a space complexity of: $O(1)$: Because the amount of data we are storing is not proportional and independent of the input size.

Note: We could of also solved this using a full circle with `randrange`.

EXERCISE: APPROXIMATE AN INTEGRAL WITH MONTE CARLO METHODS

Think of trying to find an integral from a to b for $f(x)$ as trying to find the area under the curve of $f(x)$ from $x_1 = a$ to $x_2 = b$.

- **Area of a rectangle:** $a_r = \text{length} * \text{width}$

$$f(x) = \sin(x)$$

We want to try and approximate $\int_0^\pi f(x)\,\mathrm{d}x$. So what we are saying is: "We want the area under the curve from $x_1 = 0$ to $x_2 = \pi$ for the curve $f(x)$."

We know that $-1 \leq \sin(x) \leq 1$, but since we're dealing with the first quadrant ($x \in [0, \pi]$), we will only consider positive values: $0 \leq \sin(x) \leq 1$.

To use Monte Carlo methods in geometry, we need the proportion of points in a shape inside another shape, so let's consider a rectangle around the curve.

Let's consider the rectangle:

- **width**: $w = x_2 - x_1 = \pi - 0 = \pi$.
- **height**: $h = 1$ (since $0 \leq \sin(x) \leq 1$)

The area of the rectangle would be $a_t = w * h = \pi$.

Now, let's generate n amount of random points with random x and y's, and check if they are under(and in this case inside) the curve or not.

A random point $P(x_p, y_p)$ within the rectangle is only going to be under $f(x)$ if: $y_p \leq f(x_p)$.

Let's call the area of the curve: a_c from now on.

Because the proportion of points inside the curve approximates the proportion of the curve's area relative to the rectangle's area. Then we can derive this formula:

$$\frac{\text{points in the curve}}{\text{points in the rectangle}} \approx \frac{\text{area of the curve}}{\text{area of the rectangle}}$$

$$\frac{\text{points in the curve}}{\text{points in the rectangle}} \approx \frac{a_c}{a_t}$$

And since we want the area of the curve (a_c):

$$a_c \approx \frac{\text{points in the curve}}{\text{points in the rectangle}} * a_t$$

Here are the visuals: https://www.desmos.com/calculator/um9twf3ven

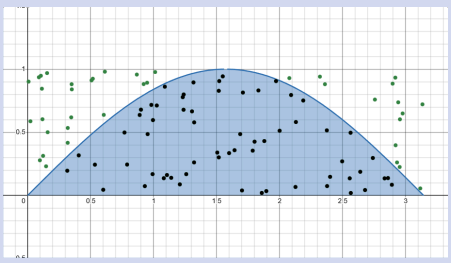

Figure 5: $n = 100$

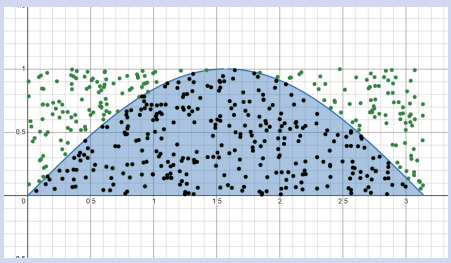

Figure 6: $n = 500$

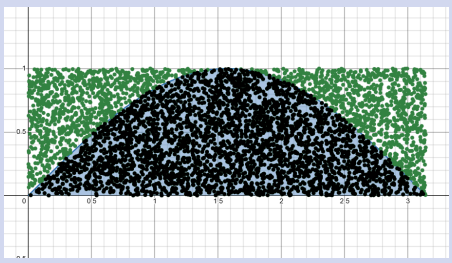

Figure 7: $n = 5000$

1. Write a multi-step strategy towards solving the problem.
2. Write an algorithm to solve this problem.
3. Describe the upper-bound time and space complexity of your solution.

SOLUTION:

1.
   - First, since we are using `sin` and `random`, we need to import the `random` and `math` libraries.
   - Then, we need to define the function `f(x)`.
   - After that we must make a function for our solution that accepts the parameter: `n`, which is the number of points to be generated randomly.
   - We need an accumulator variable to register how many points went *in the curve*, we will call it `points_in_curve`.
   - Then we could define our rectangle and get it's area.
   - Then we loop `range(n)`.
   - For each iteration, we will generate a random y between 0 and 1, and a random x between 0 and π (We will do that my multiplying the random number between 0 and 1 by π).
   - Then we will use $y \leq f(x)$ to check if the point is in the curve.
   - If it is in the curve, we will increment the `points_in_curve` accumulator.
   - Then, we use the derived formula $\frac{\text{points in the curve}}{\text{points in the rectangle}} * \text{rectangle area}$ and return the result to approximate the integral.
2.
```python
import math
import random

def f(x):
    return math.sin(x)

def solution(n):
    points_in_curve = 0

    rectangle_height = 1
    rectangle_width = math.pi
    rectangle_area = rectangle_height * rectangle_width

    for i in range(n):
        x = random.random() * rectangle_width
        y = random.random() * rectangle_height

        if y <= f(x):
            points_in_curve += 1

    return (points_in_curve / n) * rectangle_area
```
3. This algorithm has a time complexity of: $O(n)$: Because instructions increase linearly with input size. And has a space complexity of: $O(1)$: Because the amount of data we are storing is not proportional and independant of the input size.

EXERCISE: APPROXIMATE THE AREA OF AN ELLIPSOID USING MONTE CARLO METHODS

An ellipsoid is a 3-D shape which is the analogue of an ellipse.

- **Formula for an ellipsoid centered** $(0,0)$: $\frac{x^2}{a^2}+\frac{y^2}{b^2}+\frac{z^2}{c^2}=1$, where a represents the x-stretch, b the y-stretch, and z the z-stretch.
- **Point** $P(x_p, y_p, z_p)$ **inside an ellipsoid**: $\frac{x^2}{a^2}+\frac{y^2}{b^2}+\frac{z^2}{c^2}\leq 1$
- **Point** $P(x_p, y_p, z_p)$ **outside an ellipsoid**: $\frac{x^2}{a^2}+\frac{y^2}{b^2}+\frac{z^2}{c^2}> 1$
- **Volume of a cuboid**: $a_c = \text{width} * \text{height} * \text{length}$, where l is the length, w is the width, and h is the height.

For this exercise, we will take $a = 2.5 \text{ units}$, $b = 1.5 \text{ units}$, $c = 1 \text{ unit}$ for our ellipsoid.

To use Monte Carlo methods in geometry, we need the proportion of points in a shape inside another shape, so let's consider a cuboid(3-D rectangle) around the ellipsoid.

Now let's figure out the cuboid:

- **width**: $w = 2 * a = 2 * 2.5 = 5$
- **length**: $l = 2 * b = 2 * 1.5 = 3$
- **height**: $h = 2 * c = 2 * 1 = 2$

Since stretch is in both directions, we multiply every stretch factor by 2.

This time we will not constrain our self with values from 0 to 1, since both the width and the height are ≥ 1.

Now let's figure out the area:

$$a_c = w * h * l = 5 * 3 * 2 = 30 \text{ units}$$

Let's now consider a_e as the volume of the ellipsoid, and compute $\frac{a_e}{a_c}$:

$$\frac{a_e}{a_c} = \frac{a_e}{30}$$

Now, we generate n amounts of points with random x, y's and z's, and check if they are in the ellipsoid to derive the formula:

$$\frac{\text{points in the ellipsoid}}{\text{points in the cuboid}} \approx \frac{a_e}{30}$$

$$a_e \approx 30 * \frac{\text{points in ellipsoid}}{\text{points in cuboid}}$$

Here are the visuals: https://www.desmos.com/3d/4fqfrfahxc

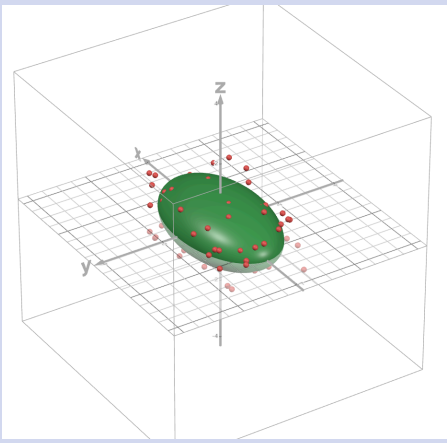

Figure 8: $n = 100$

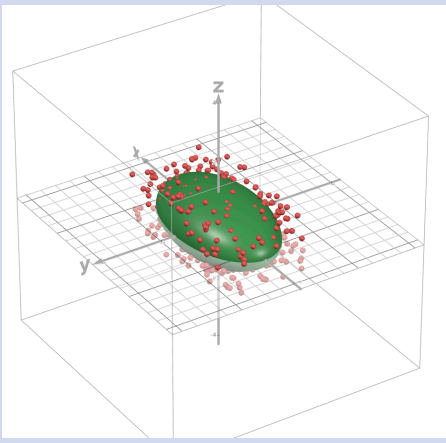

Figure 9: $n = 500$

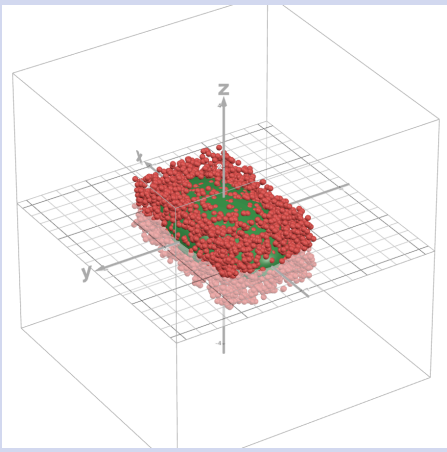

Figure 10: $n = 5000$

1. Write a multi-step strategy towards solving the problem.
2. Write an algorithm to solve this problem.
3. Describe the upper-bound time and space complexity of your solution.

SOLUTION:

1.
   - First we import the random library to get the random values.
   - Then we make the function that accepts the parameter n, the number of random points to be generated.
   - We make an accumulator variable `points_in_ellipsoid`, that counts how many points are in the circle.
   - We will register variables for the x-stretch, y-stretch and z-stretch so that we can use them later in the formula.
   - We will then loop over `range(n)`.
   - For each loop, we will generate a random x, y, and z value between $]-a, a[$, $]-b, b[$ and $]-c, c[$ respectively, using `random.uniform`.
   - We will then apply the formula $\frac{x^2}{a^2} + \frac{y^2}{b^2} + \frac{z^2}{c^2} \leq 1$ to check if the point is inside the circle.
   - If the point is inside the circle, we will increment the `points_in_ellipsoid` by 1.
   - Then, since the formula $30 * \frac{\text{points in ellipsoid}}{\text{points in cuboid}}$ is already given to us, we can just apply it and return the result.
2.

```python
import random

def solution(n):
    points_in_ellipsoid = 0

    a = 2.5
    b = 1.5
    c = 1

    for i in range(n):
        x = random.uniform(-a, a)
        y = random.uniform(-b, b)
        z = random.uniform(-c, c)

        if ((x**2) / (a**2)) + ((y**2) / (b**2)) + ((z**2) / (c**2)) <= 1:
            points_in_ellipsoid += 1

    return (points_in_ellipsoid / n) * 30
```

,

3. This algorithm has a time complexity of: $O(n)$: Because instructions increase linearly with input size. And has a space complexity of: $O(1)$: Because the amount of data we are storing is not proportional and independant of the input size.

EXERCISE: ESTIMATE THE AREA OF A SUPERELLIPSE USING MONTE CARLO METHODS

Equation for a superellipse: $|\frac{x}{a}|^n + |\frac{y}{b}|^n = 1$, where a is the x-stretch, and b is the y-stretch.
Point $P(x_p, y_p)$ inside the superellipse: $|\frac{x}{a}|^n + |\frac{y}{b}|^n \leq 1$
Point $P(x_p, y_p)$ outside the superellipse: $|\frac{x}{a}|^n + |\frac{y}{b}|^n > 1$
Area of a rectangle: $a_r = \text{length} * \text{width}$

Take $a = 1$, $b = 2$ and $n = 1.5$ for the superellipse.

Find the area of the superellipse using monte carlo methods.

1. Write a multi-step strategy towards solving the problem.
2. Write an algorithm to solve this problem.
3. Describe the upper-bound time and space complexity of your solution.

SOLUTION:

1. Before writing our strategy, we need to derive the formula for the area of the superellipse in terms of the points in the superellipse, points in the rectangle and area in the rectangle.

We need to inscribe the superellipse in the rectangle, so we can get these dimensions:

- **width**: $w = 2a = 2(1) = 2$
- **height**: $h = 2b = 2(2) = 4$

We now need the area of that rectangle: $a_r = \text{width} * \text{height} = 2 * 4 = 8$

Now we can consider the area of the superellipse as a_s and compute: $\frac{a_s}{a_r} = \frac{a_s}{8}$.

Now we can derive the equation:

$$\frac{a_s}{8} \approx \frac{\text{points in superellipse}}{\text{points in the rectangle}}$$

$$a_s \approx \frac{\text{points in the superellipse}}{\text{points in the rectangle}} * 8$$

Now we can make the strategy:

- First we need to import the `math` and `random` libraries since we need their functionalities.
- Then, we make the `solution` function, that accepts `number_of_points`, which is the number of random points to be generated.
- Then, we make our accumulator variable `points_in_superellipse`.
- Then we can define our constants a, b, and n.
- Then we can loop over `range(n)` and generate a random x_p and y_p each time.
- Then we can compute the formula $|\frac{x}{a}|^n + |\frac{y}{b}|^n \leq 1$, for each point.
- If the point is inside the superellipse, then we can increment the accumulator, otherwise do nothing.
- In the end, we can return our derived formula $\frac{\text{points in the superellipse}}{\text{points in the rectangle}} * 8$.

2.
```python
import random
import math

def solution(number_of_points):
    points_in_superellipse = 0

    a = 1
    b = 2
    n = 1.5
    rectangle_area = 8

    for i in range(number_of_points):
        x_p = random.uniform(-a, a)
        y_p = random.uniform(-b, b)

        distance = (math.fabs(x_p/a) ** n) + (math.fabs(y_p/b) ** n)

        if distance <= 1:
            points_in_superellipse += 1

    return (points_in_superellipse / number_of_points) * rectangle_area
```

3. This algorithm has a time complexity of: $O(n)$: Because instructions increase linearly with input size. And has a space complexity of: $O(1)$: Because the amount of data we are storing is not proportional and independant of the input size.